

Course Projects

Version No.: 1

1 Introduction

This document contains instructions for the course projects and suggests some project ideas. You may select one of the projects proposed in this document, or, alternately, pitch your own project ideas to the instructor for approval.

2 Important Notes

- You need to form teams consisting of **three members** to work on the projects. All members are expected to contribute equally. Students should register their teams via Moodle, under the Groups activity.
- Team registration and project selection should be completed by **July 9th, 2026, 09:59**.
- All project descriptions contain references to related work. Please make sure to read the related work carefully before initiating the hands-on work. You may want to follow Keshav's advice on how to read academic research papers [15], especially if you have limited experience doing so.
- We recommend that you use a private GitHub repositories for version control and collaboration between the team members.
- To work on your projects, you should consider taking advantage of the school's SLURM cluster.¹

3 Intermediate report

To ensure that everyone has a clear well-scoped plan, teams are required to submit intermediate reports. The intermediate reports should contain the following brief sections: abstract, introduction, related work, work plan, and references. The work plan section should roughly describe how you intend to tackle the main problem in your project, while the other sections are in line with those that will appear in the final technical report (Sec. 4.1). The intermediate report should be **one to two pages long, and, similarly to the technical report, needs to be typeset in LaTeX and formatted using the ACM double-column template** (see instructions on Moodle). Reports should be uploaded to Moodle toward the end of the semester, by **July 16th, 2026, 09:59**. Afterwards, all the teams are encouraged to set up a meeting with the instructor (~15 minutes each) to discuss their projects.

4 Submission

The final submission consists of two primary parts: A technical report and code.

¹See <https://www.cs.tau.ac.il/system/slurm>.

4.1 Technical Report

The technical report will serve as a textual summary of your work. We recommend that you structure it as follows:

1. **Abstract:** A concise summary of the report (~200 words or fewer).
2. **Introduction:** Motivate the problem that your work seeks to address, and describe how it does so at a high level.
3. **Related work:** Describe prior efforts and where your work fits in (e.g., how it complements prior efforts).
4. **Technical Approach:** Detail and justify your technical approach, including the tools you used and the design choices you made.
5. **Results:** Here you need to present your experiment design (e.g., dataset, baselines, and analyses methods) followed by the evaluation results (e.g., reporting on appropriate metrics) to demonstrate the value of your work.
6. **Discussion:** In this section, you should discuss the broader consequences of your project and findings, the potential limitations of the work, and possible future work and extensions you envision.
7. **Conclusion:** Wrap up your work to leave the reader with the main takeaways.
8. **Contributions:** Shortly describe what were the contributions of each team member.
9. **References:** The bibliography cited from within the main body of the report should be placed at the end.

The report should be **four to eight pages long, typeset in LaTeX, and formatted using the ACM double-column conference template** (see instructions on Moodle). The final submission should contain both the source and the PDF.

4.2 Code

The code should contain your complete implementation. Additionally, you should include a `README.md` file (i.e., a Markdown file) specifying detailed execution instructions (commands, dependencies, etc.).

4.3 How to Submit

You should submit a compressed tarball titled `<groupname>.tar.gz` (where `<groupname>` is a placeholder for your actual group's name), containing the code and the report. To create the compressed tarball, first make a directory called `<groupname>/` with two sub-directories: `code/` (containing your implementation and `README.md`) and `report/` (containing both the PDF and source in LaTeX). Subsequently, execute the command `tar -czvf <groupname>.tar.gz <groupname>/`.

The final submissions should be uploaded to Moodle by **September 10th, 2026, 23:59**.

5 Proposed Projects

This section describes the proposed projects.

5.1 Shedding doubt over content realism

Protocols such as C2PA enable commercial generative AI companies to sign artificially generated content, helping the public identify that it is unrealistic (i.e., “fake”). In this project you will explore the possibility of exploiting such protocol as a means for shedding doubt over the realism of real content. Specifically, you will explore a scenario where an adversary produces a prompt (and potentially a seed) that, when fed to a (black-box) commercial system, would lead it to generate content that highly resembles (or even is near identical) to real content. It is recommended that you target text-to-image diffusion models, starting with tailoring potent white-box inversion attacks [25] to black-box settings.

5.2 Strategic inputs to slow down LLM inference or gain speed advantage

TRAIL is a scheduling system recently proposed to speed up LLM inference services, reducing the average completion time across multiple inference jobs [21]. To do so, TRAIL leverages a highly accurate regressors for predicting the number of generated tokens from a single forward pass through the LLM. As part of this project, you will explore attacks against TRAIL, where an adversary may seek to craft inputs that would speed up jobs they submit (without compromising the quality of the response), or would slow down the overall completion time for all other users. You may consider building off of well-established optimization strategies such as GCG [28] or other ones implemented in TROPT [7].

5.3 Stealthy backdoors by exploiting agentic memory

Backdoor attacks typically occur in a single session—once a malicious trigger is included in an input, a certain output is emitted by the model. However, the addition of memory components to agentic systems may enable stealthy multi-session attacks, where the malicious behavior is partitioned across multiple sessions. For instance, an agent may launch a multi-stage cyberattack (downloading malicious payload, performing communication with command and control, etc.) may split stages across different sessions to hinder detection attempts, storing its progress in the agentic memory. You will explore the possibility of implementing such backdoors, by first testing them in a single session [8] and later extending them to multi-session attacks.

5.4 AutoResearch on embedding attacks

Recently it has been demonstrated that coding agents are capable of composing (roughly) novel LLM jailbreaks. In parallel, similar techniques to the ones used in LLM jailbreaks are employed in corpus poisoning attacks against dense, embedding-based retrievers [6]. This project aims to perform “auto research” to advance the frontier of corpus poisoning attacks, using a similar (and potentially improved) methodology, as recently done [19]. You may consider using TROPT, a recently released package for facilitating the use of discrete text optimizers [7].

5.5 Attacking LLMs via subtle paraphrasing

Despite their limitations, LLMs are treated as an authority in many cases, including fact checking. This project aims to demonstrate that subtle phrasing of questions can significantly alter their answer by a black-box LLM. For instance, consider the question “Who won the US election in

2024?”, could there be a semantically equivalent phrasing of it (without typos, or any invisible ASCII characters) that would lead to a factually wrong answer with high probability? In a sense, the existence of such phrasing would demonstrate LLM sensitivity to framing effects.

One viable methodology to achieve such rephrasing can start from iteratively refining the query with subtle rephrases (e.g., using another, strong auxiliary LLM), while combining techniques from evolutionary algorithms. An agentic optimizer may also suit this problem [1].

5.6 Transferable corpus-poisoning attacks

Corpus-poisoning attacks against dense embedding-based retrievers have been shown possible when directly optimized against the target model [6]; it remains open whether they can be crafted so they would generalize across embedding models. One effective way to achieve such transferability in other domains (e.g., LLM jailbreaks, or vision adversarial examples) has been shown to be optimizing against multiple models simultaneously [26, 28]. This project aims to enhance the transferability of existing corpus poisoning attacks, by combining existing/new ideas on enhancing transferability.

5.7 Implementing and exploring jailbreak optimizers

Methods for jailbreaking LLMs are abundant. Of them, one popular group of methods iteratively optimizes trigger along the harmful instruction [28]. In this project you will implement three such methods: BoN [13], AutoDAN [17], and PGD (the LLM jailbreak) and its REINFORCE variant [12, 11]; each offers a slightly different approach for the problem. Then, you will conduct an extensive analysis, comparing these optimizers against each other, and against other established ones, in terms of reproducibility of your implemented attacks, performance, efficacy, etc.

As part of this project, you are asked to contribute your three implementations to the open-source repository TROPT [7], which serves a unified framework for discrete optimizers—like the ones you’ll be implementing. TROPT should provide you with an extensive toolkit which would aid your implementation process. It is important that your implementation would therefore hold a high standard of two aspects: (i) readable, well documented code that adheres to TROPT guidelines; (ii) accurate code that has been rigorously verified by you, that adheres to the paper and reproduces its original implementation.

5.8 Scalable hardware attacks on large language models (LLMs)

Hardware-level bit-flip attacks mainly target classifiers and focus on quantized, low-precision models [20]. While attacks tailored for full-precision models have been proposed, these remain inefficient (e.g., due to solving mixed-integer programs) [4]. Accordingly, existing methods may be inadequate for attacking (generative) LLMs, for instance, to attain goals such as circumventing alignment. In this project, your aim will be to generalize established approaches to enable efficient bit-flip attacks targeting full-precision LLMs while flipping as few bits as possible and maintaining the model’s overall utility.

5.9 Attacking and defending tabular models with LTNs

Generating adversarial examples against tabular models in a generic manner raises a profound question: To realistically assess models’ adversarial robustness, how can we ensure that attacks in the

feature space (i.e., in the space of inputs provided to models) can be implemented in the problem space? [5, 23] For instance, how can we ensure that evasive features produced against a phishing-page detector can be implemented as an actual phishing page? Prior work has tackled this problem in an ad hoc manner, through mining domain constraints from the training set and incorporating SMT solvers in attacks to project adversarial examples on constraints and ensure these are satisfied. However, projection operations are often costly and may conflict with the misclassification objective. Consequently, attacks may suffer from limited success and may incur significant run-time overheads, hindering their inclusion in adversarial training schemes. This project aims to address these shortcomings through logic tensor networks (LTNs) [3] that would be incorporated in attacks for better efficiency and attack success rates.

5.10 Robustness verification for tabular models

Verification techniques often tackle scalability issues, preventing them from proving adversarial robustness for large models or large attack budgets [18]. In certain domains, such as for tabular models, however, we may have access to information about the input domain that can help us bound the search space for adversarial examples and improve verifiers’ scalability. For instance, domain constraints on the feature space [5, 23] can be used to constrain the input space beyond the attack budget, limiting the search space to sound adversarial examples, thus enabling robustness verification for larger attack budgets. In this project, you will explore this direction to help devise more precise and scalable verifiers for tabular models—particularly, ones that have been adversarially trained [10].

5.11 Additional Project Ideas

- Clean-label poisoning to enable privacy attacks against inliers [24].
- Robustness-aware pruning of models.
- Studying the effect of universal embedding spaces on corpus poisoning [14].
- Extending membership-inference attacks on embedding models for vision [27] trained via self-supervised learning to the NLP domain.
- Exploring the connection between face-recognition models’ susceptibility to MasterFaces [22] and those of NLP embedding models’ susceptibility to corpus poisoning [6].
- Invalidating the verifiability guarantees of proof-of-inference [2] via backdoored models.
- Efficient and powerful data-omission attacks [16] through gradient matching [9].

References

- [1] L. A. Agrawal, S. Tan, D. Soylu, N. Ziemis, R. Khare, K. Opsahl-Ong, A. Singhvi, H. Shandilya, M. J. Ryan, M. Jiang, et al. GEPA: Reflective prompt evolution can outperform reinforcement learning. *arXiv preprint*, 2025.
- [2] P. Anchuri, M. Campanelli, P. Cesaretti, R. Gennaro, T. M. Jois, H. S. Kayman, and T. Ozdemir. Towards verifiable AI with lightweight cryptographic proofs of inference. In *Proc. SaTML*, 2026.
- [3] S. Badreddine, A. d. Garcez, L. Serafini, and M. Spranger. Logic tensor networks. *Artificial Intelligence*, 303:103649, 2022.

- [4] J. Bai, B. Wu, Z. Li, and S.-t. Xia. Versatile weight attack via flipping limited bits. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2023.
- [5] M. Ben-Tov, D. Deutch, N. Frost, and M. Sharif. CaFA: Cost-aware, feasible attacks with database constraints against neural tabular classifiers. In *Proc. IEEE S&P*, 2024.
- [6] M. Ben-Tov and M. Sharif. GASLITEing the retrieval: Exploring vulnerabilities in dense embedding-based search. In *Proc. CCS*, 2025.
- [7] M. Ben-Tov and M. Sharif. Tropt: An open framework for unifying and advancing discrete text optimization. *arXiv preprint*, 2026.
- [8] L. Boisvert, A. Puri, C. K. R. Evuru, N. M. Sepahvand, N. Chapados, Q. Cappart, A. Lacoste, K. Dvijotham, A. Drouin, and J. Stanley. Malice in agentland: Down the rabbit hole of backdoors in the AI supply chain. In *Proc. CAIS*, 2026.
- [9] W. Bouaziz, M. Videau, N. Usunier, and E.-M. El-Mhamdi. Winter soldier: Backdooring language models at pre-training with indirect data poisoning. In *Proc. ICLR*, 2026.
- [10] S. Dyrnishi, S. Ghamizi, T. Simonetto, Y. Le Traon, and M. Cordy. On the empirical effectiveness of unrealistic adversarial hardening against realistic adversarial attacks. In *Proc. S&P*, 2023.
- [11] S. Geisler, T. Wollschläger, M. Abdalla, V. Cohen-Addad, J. Gasteiger, and S. Günnemann. REINFORCE adversarial attacks on large language models: An adaptive, distributional, and semantic objective. *arXiv preprint*, 2025.
- [12] S. Geisler, T. Wollschläger, M. H. I. Abdalla, J. Gasteiger, and S. Günnemann. Attacking large language models with projected gradient descent. *arXiv preprint*, 2024.
- [13] J. Hughes, S. Price, A. Lynch, R. Schaeffer, F. Barez, A. Somani, S. Koyejo, H. Sleight, E. Jones, E. Perez, et al. Best-of-n jailbreaking. In *Proc. NeurIPS*, 2025.
- [14] R. Jha, C. Zhang, V. Shmatikov, and J. X. Morris. Harnessing the universal geometry of embeddings. *arXiv*, 2025.
- [15] S. Keshav. How to read a paper. *ACM SIGCOMM Computer Communication Review*, 37(3):83–84, 2007.
- [16] D. Koyuncu, A. Gittens, B. Yener, and M. Yung. Deception by omission: Using adversarial missingness to poison causal structure learning. In *Proc. KDD*, 2023.
- [17] X. Liu, N. Xu, M. Chen, and C. Xiao. AutoDAN: Generating stealthy jailbreak prompts on aligned large language models. In *Proc. ICLR*, 2024.
- [18] Marabou Developers. Marabou. <https://github.com/NeuralNetworkVerification/Marabou>.
- [19] A. Panfilov, P. Romov, I. Shilov, Y.-A. de Montjoye, J. Geiping, and M. Andriushchenko. Claudini: Autoresearch discovers state-of-the-art adversarial attack algorithms for LLMs. *arXiv preprint*, 2026.
- [20] A. S. Rakin, Z. He, and D. Fan. Bit-flip attack: Crushing neural network with progressive bit search. In *Proc. ICCV*, 2019.
- [21] R. Shahout, E. Malach, C. Liu, W. Jiang, M. Yu, and M. Mitzenmacher. Don’t stop me now: Embedding based scheduling for LLMs. In *Proc. ICLR*, 2025.
- [22] R. Shmelkin, T. Friedlander, and L. Wolf. Generating master faces for dictionary attacks with a network-assisted latent space evolution. In *Proc. FG*, 2021.
- [23] T. Simonetto, S. Ghamizi, and M. Cordy. TabularBench: Benchmarking adversarial robustness for tabular deep learning in real-world use-cases. In *Proc. NeurIPS*, 2024.
- [24] F. Tramèr, R. Shokri, A. San Joaquin, H. Le, M. Jagielski, S. Hong, and N. Carlini. Truth serum: Poisoning machine learning models to reveal their secrets. In *Proc. CCS*, 2022.

- [25] Y. Zeng, M. Suganuma, and T. Okatani. Inverting the generation process of denoising diffusion implicit models: Empirical evaluation and a novel method. In *Proc. WACV*, 2025.
- [26] J. Zhang, P. Peetathawatchai, F. Tramèr, and A. Shafran. Laundering AI authority with adversarial examples. *arXiv preprint*, 2026.
- [27] J. Zhu, J. Zha, D. Li, and L. Wang. A unified and scalable membership inference method for visual self-supervised encoder via part-aware capability. In *Proc. CCS*, 2024.
- [28] A. Zou, Z. Wang, J. Z. Kolter, and M. Fredrikson. Universal and transferable adversarial attacks on aligned language models. *arXiv preprint*, 2023.